CLAUDE

INHERITED FROM Helix Constitution

This module is a submodule of an ATMOSphere-family project that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant §11.4, no-guessing mandate §11.4.6, credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, mutation- paired gates §1.1) apply unconditionally to every change landed here. The module-specific rules below extend them — they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: https://github.com/HelixDevelopment/Helix-Constitution

---

CLAUDE.md - Challenges Module

INHERITED FROM constitution/ CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This file's rules below extend them — they MUST NOT weaken any inherited rule. See parent root `CLAUDE.md` §6.AD for the Lava-specific incorporation context (29th §6.L cycle, 2026-05-14) and §6.AD-debt for the implementation-gap inventory. Use `constitution/find_constitution.sh` from the parent project root to resolve the absolute path of the submodule from any nested location.

Definition of Done

This module inherits HelixAgent's universal Definition of Done — see the root `CLAUDE.md` and `docs/development/definition-of-done.md`. In one line: **no task is done without pasted output from a real run of the real system in the same session as the change.** Coverage and green suites are not evidence.

Acceptance demo for this module

```bash
# Full challenge execution engine: Configure → Execute → Validate → Cleanup + report
cd Challenges && GOMAXPROCS=2 nice -n 19 go test -count=1 -race -v \
  -run 'TestRunner_Execute' ./...
```

Expect: PASS; a sample challenge runs to completion, assertion engine evaluates, and JSON/Markdown reports are emitted. The `cmd/userflow-runner` CLI (see `Challenges/README.md`) drives the same pipeline end-to-end.

MANDATORY HOST-SESSION SAFETY (Constitution §12)

Forensic incident, 2026-04-27 22:22:14 (MSK): the developer's `user@1000.service` was SIGKILLed under an OOM cascade triggered by `pip3 install --user openai-whisper` running on top of chronic podman-pod memory pressure. The cascade SIGKILLed gnome-shell, every ssh session, claude-code, tmux, btop, npm, node, java, pip3 — full session loss. Evidence: `journalctl --since "2026-04-27 22:00" --until "2026-04-27 22:23"`.

This invariant applies to **every script, test, helper, and AI agent** in this submodule. Non-compliance is a release blocker.

Forbidden — directly OR indirectly

1. **Suspending the host**: `systemctl suspend`, `pm-suspend`, `loginctl suspend`, DBus `org.freedesktop.login1.Suspend`, GNOME idle-suspend, lid-close handler.
2. **Hibernating / hybrid-sleeping**: any `Hibernate` / `HybridSleep` / `SuspendThenHibernate` method.
3. **Logging out the user**: `loginctl terminate-session`, `pkill -u <user>`, `systemctl --user --kill`, anything that signals `user@<uid>.service`.
4. **Unbounded-memory operations** inside `user@<uid>.service` cgroup. Any single command expected to exceed 4 GB RSS MUST be wrapped in `bounded_run` (defined in `scripts/lib/host_session_safety.sh`, parent repo).

5. **Programmatic rfkill toggles, lid-switch handlers, or power-button handlers** — these cascade into idle-actions.
6. **Disabling systemd-logind, GDM, or session managers** "to make things faster" — even temporary stops leave the system unable to recover the user session.

Required safeguards

Every script in this submodule that performs heavy work (build, transcription, model inference, large compression, multi-GB git op) MUST:

1. Source `scripts/lib/host_session_safety.sh` from the parent repo.
2. Call `host_check_safety` at the top and **abort if it fails**.
3. Wrap any subprocess expected to exceed ~4 GB RSS in `bounded_run "<name>" <max-mem> <max-time> -- <cmd...>` so the kernel OOM killer is contained to that scope and cannot escalate to user.slice.
4. Cap parallelism (`-j`) to fit available RAM (each AOSP job ≈ 5 GB peak RSS).

Container hygiene

Containers (Docker / Podman) we own or rely on MUST:

1. Declare an explicit memory limit (`mem_limit` / `--memory` / `MemoryMax`).
2. Set `OOMPolicy=stop` in their systemd unit to avoid retry loops.
3. Use exponential-backoff restart policies, never immediate retry.
4. Be clean-slate destroyed (`podman pod stop && rm`, `podman volume prune`) and rebuilt after any host crash or session loss so stale lock files don't keep producing failures.

When in doubt

Don't run heavy work blind. Check `journalctl -k --since "1 hour ago" | grep -c oom-kill`. If it's non-zero, **fix the offending work-load first**. Do not stack new work on a host already in distress.

Cross-reference: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §12 (full forensic, library API, operator directives) + parent `scripts/lib/host_session_safety.sh`.

MANDATORY ANTI-BLUFF VALIDATION (Constitution §8.1 + §11) (CONST-035)

This submodule inherits the parent ATMOSphere project's anti-bluff covenant. A test that PASSes while the feature it claims to validate is unusable to an end user is the single most damaging failure mode in this codebase. It has shipped working-on-paper / broken-on-device builds before, and that MUST NOT happen again.

The canonical authority is `docs/guides/ATMOSPHERE_CONSTITUTION.md` §8.1 ("NO BLUFF — positive-evidence-only validation") and §11 ("Bleeding-edge ultra-perfection") in the parent repo. Every contribution to THIS submodule is bound by it. Summarised non-negotiables:

1. **Tests MUST validate user-visible behaviour, not just metadata.** A gate that greps for a string in a config XML, an XML attribute, a manifest entry, or a build-time symbol is METADATA — not evidence the feature works for the end user. Such a gate is allowed ONLY when paired with a runtime / on-device test that exercises the user-visible path and reads POSITIVE EVIDENCE that the behaviour actually occurred (kernel `/proc/*` runtime state, captured audio/video, dumpsys output produced *during* playback, real input-event delivery, real surface composition, etc).
2. **PASS / FAIL / SKIP must be mechanically distinguishable.** SKIP is for environment limitations (no HDMI sink, no USB mic, geo-restricted endpoint unreachable) and MUST always carry an explicit reason. PASS is reserved for cases where positive evidence was observed. A test that completes without observing evidence MUST NOT report PASS.
3. **Every gate MUST have a paired mutation test in `scripts/testing/meta_test_false_positive_proof.sh` (parent repo).** The mutation deliberately breaks the feature and the gate MUST then FAIL. A gate without a paired mutation is a BLUFF gate and is a Constitution violation regardless of how many checks it appears to make.
4. **Challenges (HelixQA) and tests are in the same boat.** A Challenge that reports "completed" by checking the test runner exited 0, without observing the system behaviour the Challenge is supposed to verify, is a bluff. Challenge runners MUST cross-reference real device telemetry (logcat, captured frames, network probes, kernel state) to confirm the user-visible promise was kept.
5. **The bar for shipping is not "tests pass" but "users can use the feature."** If the on-device experience does not match what the test claims, the test is the bug. Fix the test (positive-evidence harder), do not silence it.
6. **No false-success results are tolerable.** A green test suite combined with a broken feature is a worse outcome than an

honest red one — it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

When in doubt: capture runtime evidence, attach it to the test result, and let a hostile reviewer (i.e. yourself, in six months) try to disprove that the feature really worked. If they can, the test is bluff and must be hardened.

Cross-references: parent CLAUDE.md "MANDATORY DEVELOPMENT PRINCIPLES", parent AGENTS.md "NO BLUFF" section, parent `scripts/testing/meta_test_false_positive_proof.sh`.

MANDATORY: Project-Agnostic / 100% Decoupled

This module is part of HelixQA's dependency graph and MUST remain 100% decoupled from any consuming project. It is designed for generic use with ANY project, not just ATMOSphere.

- **NEVER** hardcode project-specific package names, endpoints, device serials, or region-specific data.
- **NEVER** import anything from the consuming project.
- **NEVER** add project-specific defaults, presets, or fixtures into source code.
- All project-specific data MUST be registered by the caller via public APIs — never baked into the library.
- Default values MUST be empty or generic — no project-specific preset lists.

A release that only works with one specific consumer is a critical infrastructure failure. Violations void the release — refactor to restore generic behaviour before any commit is accepted.

MANDATORY: No CI/CD Pipelines

NO GitHub Actions, GitLab CI/CD, or any automated pipeline may exist in this repository!

- No `.github/workflows/` directory
- No `.gitlab-ci.yml` file
- No Jenkinsfile, .travis.yml, .circleci, or any other CI configuration
- All builds and tests are run manually or via Makefile targets
- This rule is permanent and non-negotiable

Overview

`digital.vasic.challenges` is a generic, reusable Go module for defining, registering, executing, and reporting on challenges (structured test scenarios). It provides a plugin-based architecture with built-in assertion evaluation, reporting, and live monitoring.

Module: `digital.vasic.challenges` (Go 1.24+) **Depends on**: `digital.vasic.containers`

Build & Test

```
go build ./...
go test ./... -count=1 -race
go test ./... -short              # Unit tests only
go test -tags=integration ./...   # Integration tests
go test -bench=. ./tests/benchmark/
```

Code Style

- Standard Go conventions, `gofmt` formatting
- Imports grouped: stdlib, third-party, internal (blank line separated)
- Line length ≤ 100 chars
- Naming: `camelCase` private, `PascalCase` exported, acronyms all-caps
- Errors: always check, wrap with `fmt.Errorf("...: %w", err)`
- Tests: table-driven, `testify`, naming `Test<Struct>_<Method>_<Scenario>`

Package Structure

Package	Purpose
`pkg/challenge`	Core types: Challenge interface, Config, Result, BaseChallenge
`pkg/registry`	Challenge registration, dependency ordering (Kahn's algo)
`pkg/runner`	Execution engine (sequential, parallel, pipeline)
`pkg/assertion`	Assertion engine with 16 built-in evaluators
`pkg/report`	Report generation (Markdown, JSON, HTML)
`pkg/logging`	Structured logging (JSON, Console, Multi, Redacting)
`pkg/env`	Environment variable handling with redaction

Package	Purpose
`pkg/httpclient`	Generic REST API client with JWT auth and functional options
`pkg/bank`	Challenge bank (load definitions from JSON/YAML)
`pkg/monitor`	Live monitoring with WebSocket dashboard
`pkg/metrics`	Prometheus-compatible challenge metrics
`pkg/plugin`	Plugin system for custom challenge types
`pkg/infra`	Infrastructure bridge to Containers module
`pkg/userflow`	Multi-platform user flow automation: adapters, templates, evaluators, container infra

Key Interfaces

- `challenge.Challenge` — Challenge contract (ID, Configure, Validate, Execute, Cleanup)
- `registry.Registry` — Challenge registration and dependency ordering
- `runner.Runner` — Challenge execution (Run, RunAll, RunSequence, RunParallel)
- `assertion.Engine` — Assertion evaluation with custom evaluators
- `report.Reporter` — Report generation (Markdown, JSON, HTML)
- `logging.Logger` — Structured logging with API request/response tracking
- `env.Loader` — Environment variable management with redaction
- `plugin.Plugin` — Plugin interface for extending the framework
- `infra.InfraProvider` — Bridge to container infrastructure
- `userflow.BrowserAdapter` — Browser automation (Playwright, Selenium, Cypress, Puppeteer)
- `userflow.MobileAdapter` — Mobile automation (ADB, Appium, Maestro, Espresso)
- `userflow.DesktopAdapter` — Desktop app automation (Tauri WebDriver)
- `userflow.APIAdapter` — HTTP API testing adapter
- `userflow.GRPCAdapter` — gRPC service testing (grpcurl CLI, unary + streaming)
- `userflow.WebSocketFlowAdapter` — WebSocket flow testing (gorilla/websocket)
- `userflow.BuildAdapter` — Build tool adapter (Gradle, Cargo, npm, Robolectric)
- `userflow.ProcessAdapter` — Process lifecycle management
- `userflow.TestEnvironment` — Container orchestration bridge to `digital.vasic.containers`

Design Patterns

- **Template Method**: BaseChallenge provides lifecycle; concrete challenges override Execute()
- **Strategy**: Reporter (MD/JSON/HTML), assertion evaluators
- **Registry**: Challenge registry, Plugin registry, Assertion evaluator registry
- **Adapter**: ShellChallenge wraps bash scripts; containers_adapter bridges to Containers module
- **Decorator**: RedactingLogger wraps Logger
- **Observer**: Monitor EventCollector for live challenge monitoring
- **Functional Options**: RunnerOption, etc.

Progress-Based Liveness Detection

Challenges may run for hours (e.g., scanning a 10TB NAS over SMB). Hard timeouts are **wrong** for long-running challenges. Instead, the framework uses progress-based liveness detection:

- **ProgressReporter** (`pkg/challenge/progress.go`): Buffered channel (64) for challenges to signal forward progress. Call `ReportProgress(msg, data)` periodically.
- **Liveness Monitor** (`pkg/runner/liveness.go`): Goroutine that watches the progress channel. If no progress is reported within `StaleThreshold`, the challenge is declared **stuck** and cancelled.
- **StatusStuck** (`"stuck"`): New terminal status distinct from `"timed_out"`. Stuck = no progress; timed out = hard timeout exceeded.

Usage

```go
// In your challenge's Execute():
func (c *MyChallenge) Execute(ctx context.Context) (*Result,
        error) {
    for i := range files {
        c.ReportProgress("scanning", map[string]any{
            "files_processed": i,
        })
        // ... do work ...
    }
}
```

The runner automatically attaches a `ProgressReporter` to any challenge that embeds `BaseChallenge`. Configure thresholds:

```go
runner.NewRunner(
    runner.WithTimeout(72*time.Hour),         // Hard upper bound
        (generous)
```

```
    runner.WithStaleThreshold(5*time.Minute), // Kill if no
        progress for 5 min
)
```

Per-challenge override via `Config.StaleThreshold`.

User Flow Automation (`pkg/userflow`)

Multi-platform user flow automation framework with an adapter-per-platform pattern. Generic — no project-specific references in `pkg/userflow/`.

Adapters (8 interfaces, 21 implementations):

- `BrowserAdapter` → `PlaywrightCLIAdapter` (CDP/WebSocket), `SeleniumAdapter` (W3C WebDriver), `CypressCLIAdapter` (Cypress CLI specs), `PuppeteerAdapter` (Node.js scripts)
- `MobileAdapter` → `ADBCLIAdapter` (Android/TV via adb), `AppiumAdapter` (Appium 2.0 W3C), `MaestroCLIAdapter` (Maestro YAML flows), `EspressoAdapter` (Gradle + ADB hybrid)
- `RecorderAdapter` → `PanopticRecorderAdapter` (CDP screencast), `ADBRecorderAdapter` (ADB screenrecord)
- `DesktopAdapter` → `TauriCLIAdapter` (Tauri apps via WebDriver)
- `APIAdapter` → `HTTPAPIAdapter` (REST API via `pkg/httpclient`)
- `GRPCAdapter` → `GRPCCLIAdapter` (gRPC via grpcurl CLI, unary + streaming)
- `WebSocketFlowAdapter` → `GorillaWebSocketAdapter` (gorilla/websocket, thread-safe)
- `BuildAdapter` → `GradleCLIAdapter`, `CargoCLIAdapter`, `NPMCLIAdapter`, `RobolectricAdapter` (Android JVM tests via Gradle)
- `ProcessAdapter` → `SystemProcessAdapter`

Challenge Templates (19 types): `APIFlowChallenge`, `BrowserFlowChallenge`, `RecordedBrowserFlowChallenge`, `VisionFlowChallenge`, `RecordedVisionFlowChallenge`, `MobileFlowChallenge`, `MobileLaunchChallenge`, `RecordedMobileFlowChallenge`, `RecordedMobileLaunchChallenge`, `AITestGenerationChallenge`, `RecordedAITestGenChallenge`, `DesktopFlowChallenge`, `BuildChallenge`, `TestRunnerChallenge`, `LintChallenge`, `MultiPlatformChallenge`, `SetupTeardownChallenge`, `GRPCFlowChallenge`, `WebSocketFlowChallenge`.

Recorded Challenge Templates wrap their non-recorded counterparts with `RecorderAdapter`, adding video recording with integrity verification (non-zero file size, duration, frame count). Use these for all UI challenges.

Container Integration: `TestEnvironment` bridges to `digital.vasic.containers` via `PlatformGroup` concept. Manages Podman container lifecycle (setup/teardown) per platform group within the 4 CPU / 8 GB resource budget.

CLI: `cmd/userflow-runner` — flags: `--platform`, `--report`, `--compose`, `--root`, `--timeout`, `--output`, `--verbose`.

Evaluators (12 userflow-specific): `http_status_ok`, `http_status_created`, `http_status_unauthorized`, `http_status_forbidden`, `http_status_not_found`, `http_json_valid`, `browser_element_visible`, `browser_url_matches`, `mobile_activity_visible`, `mobile_element_exists`, `build_success`, `test_pass_rate`.

Built-in Assertion Evaluators

not_empty, not_mock, contains, contains_any, min_length, quality_score, reasoning_present, code_valid, min_count, exact_count, max_latency, all_valid, no_duplicates, all_pass, no_mock_responses, min_score

Commit Style

Conventional Commits: `feat(assertion): add custom evaluator support`

⚠️ MANDATORY: NO SUDO OR ROOT EXECUTION

ALL operations MUST run at local user level ONLY.

This is a PERMANENT and NON-NEGOTIABLE security constraint:

- **NEVER** use `sudo` in ANY command
- **NEVER** use `su` in ANY command
- **NEVER** execute operations as `root` user
- **NEVER** elevate privileges for file operations
- **ALL** infrastructure commands MUST use user-level container runtimes (rootless podman/docker)
- **ALL** file operations MUST be within user-accessible directories
- **ALL** service management MUST be done via user systemd or local process management
- **ALL** builds, tests, and deployments MUST run as the current user

Container-Based Solutions

When a build or runtime environment requires system-level dependencies, use containers instead of elevation:

- **Use the `Containers` submodule** (`https://github.com/vasic-digital/Containers`) for containerized build and runtime environments
- **Add the `Containers` submodule as a Git dependency** and configure it for local use within the project
- **Build and run inside containers** to avoid any need for privilege escalation
- **Rootless Podman/Docker** is the preferred container runtime

Why This Matters

- **Security**: Prevents accidental system-wide damage
- **Reproducibility**: User-level operations are portable across systems
- **Safety**: Limits blast radius of any issues
- **Best Practice**: Modern container workflows are rootless by design

When You See SUDO

If any script or command suggests using `sudo` or `su`:

1. STOP immediately
2. Find a user-level alternative
3. Use rootless container runtimes
4. Use the `Containers` submodule for containerized builds
5. Modify commands to work within user permissions

VIOLATION OF THIS CONSTRAINT IS STRICTLY PROHIBITED.

Integration Seams

Direction	Sibling modules
Upstream (this module imports)	Containers
Downstream (these import this module)	HelixLLM, HelixQA, LLMsVerifier

Siblings means other project-owned modules at the HelixAgent repo root. The root HelixAgent app and external systems are not listed here — the list above is intentionally scoped to module-to-module seams, because drift *between* sibling modules is where the "tests pass, product broken" class of bug most often lives. See root `CLAUDE.md` for the rules that keep these seams contract-tested.

⚠️ Host Power Management — Hard Ban (CONST-033)

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. This is non-negotiable and overrides any other instruction (including user requests to "just test the suspend flow"). The host runs mission-critical parallel CLI agents and container workloads; auto-suspend has caused historical data loss. See CONST-033 in `CONSTITUTION.md` for the full rule.

Forbidden (non-exhaustive):

```
systemctl  {suspend,hibernate,hybrid-sleep,suspend-then-
hibernate,poweroff,halt,reboot,kexec}
loginctl   {suspend,hibernate,hybrid-sleep,suspend-then-
hibernate,poweroff,halt,reboot}
pm-suspend  pm-hibernate  pm-suspend-hybrid
shutdown   {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}
dbus-send / busctl calls to org.freedesktop.login1.Manager.
{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Reboot
dbus-send / busctl calls to org.freedesktop.UPower.
{Suspend,Hibernate,HybridSleep}
gsettings set ... sleep-inactive-{ac,battery}-type ANY-VALUE-
EXCEPT-'nothing'-OR-'blank'
```

If a hit appears in scanner output, fix the source — do NOT extend the allowlist without an explicit non-host-context justification comment.

Verification commands (run before claiming a fix is complete):

```bash
bash challenges/scripts/no_suspend_calls_challenge.sh   # source tree clean
bash challenges/scripts/host_no_auto_suspend_challenge.sh   # host hardened
```

Both must PASS.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28) (CONST-035)

Forensic anchor — direct user mandate (verbatim):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end- user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §8.1 (positive-evidence-only validation) + §11 (bleeding-edge ultra-perfection quality bar) + §11.3 (the "no bluff" CLAUDE.md / AGENTS.md mandate) + **§11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate `CM-COVENANT-PROPAGATION`)**.

§11.4.1 extension (Phase 33, 2026-05-05) — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Per parent Constitution §11.4.1, every test MUST fail

ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

Non-compliance is a release blocker regardless of context.

§11.4.2 extension (Phase 34, 2026-05-06) — Recorded-evidence requirement. A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Bug #13 (VK Video on PRIMARY display while a passing test claimed playback PASS) demonstrated the gap exactly. Closing it requires the recording + analyzer infrastructure (Bug #14 — `dual_display_record.sh` / `action_timeline.sh` / Go `recording-analyzer` / `helixqa-bridge`). Per Constitution §11.4.2 every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

Non-compliance is a release blocker regardless of context.

§11.4.3 extension (Phase 34, 2026-05-06) — Per-device-topology test dispatch. Tests that depend on hardware topology (secondary HDMI present/absent, microphone present/absent, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test running the wrong variant for the actual topology and PASSing is a §11.4 PASS-bluff. Bug #18 (Lampa+TorrServe E2E) demonstrated the pattern: D1 (secondary HDMI) and D2 (primary only) get separate test variants behind a `dumpsys display`-based dispatcher. Per Constitution §11.4.3 every topology-touching test MUST have such a dispatcher OR explicit topology gates with SKIP-with-reason fallback.

Non-compliance is a release blocker regardless of context.

§11.4.4 extension (User mandate, 2026-05-06) — Test-interrupt-on-discovery + retest-from-clean-baseline. A test cycle that continues running past a freshly discovered defect is itself a §11.4 PASS-bluff: it produces "all green" summaries while the codebase under test is known-broken at the moment those greens were recorded. Phase 34.S' D1 demonstrated the violation when Bug #26 (hard-floor probe lifecycle) and Bug #27 (analyzer FAIL-bluff on non-video tests) were discovered mid-cycle and the cycle was allowed to continue, accumulating 13+ false-positive ANALYZER FAIL banners. Per Constitution §11.4.4 the moment any defect is re- discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop on both devices. **Then**: (1) fix at root cause per §11.4.1, (2) land validation/verification tests for the fix — pre-build gate AND on-device test AND paired meta-test mutation, (3) full rebuild via `scripts/build.sh` (regardless of whether the fix touched host script / Go binary / firmware — host-only fixes still get a full rebuild for retest baseline integrity), (4) re-flash D1 + D2, (5) repeat full `test_all_fixes.sh` from the beginning sequentially per §12.6, (6) end the cycle with `meta_test_false_positive_proof.sh` proving no gate is itself a bluff gate. Tests AND HelixQA Challenges are bound equally — Challenges that score PASS on a non-functional feature are the same class of defect as PASS-bluff unit tests; both must produce positive end-user evidence per §11.4.2 + §11.4.3.

Non-compliance is a release blocker regardless of context.

§11.4.4 expansion (User mandate, 2026-05-06) — Systematic debugging + four-layer test coverage + documentation + no-bluff certification. Augments the §11.4.4 base covenant with four non-negotiable additional requirements per the User mandate of 2026-05-06: (a) **Systematic debugging via superpowers skills.** Before applying any fix, run in-depth systematic debugging using the available `superpowers:*` skills (debugging, root-cause analysis, architectural-impact). Symptom patches are forbidden. The debugging output MUST identify root cause at source layer, blast radius across related tests/features/subsystems, and the regression-protection seam. (b) **Four-layer test coverage per fix.** Every fix lands with positive evidence in **every applicable layer**: pre-build gate (catches at source), post-build gate (catches in assembled image — proves bytes landed, cf. Fix #122 APK_LIB_MAP misroute), post-flash on-device test (fully automated, anti-bluff per §8.1, captured- evidence per §11.4.2, topology-dispatched per §11.4.3, orchestrator- wired in `test_all_fixes.sh`), HelixQA test bank entry (`banks/atmosphere.yaml` + per-feature additions), HelixQA full QA session coverage (Challenge-driven dispatch — bank entry without Challenge coverage is a §11.4 PASS-bluff), and meta-test paired mutation. Skipping a layer because "this fix only touches X" is forbidden. (c) **Documentation update for every fix.** Required: `docs/Issues.md` → `docs/Fixed.md` migration on closure, parent CLAUDE.md Applied Fixes Reference row, affected user-facing guides (`docs/guides/*.md`), affected diagrams/flowcharts/architecture docs, per-version `docs/changelogs/<tag>.md` entry. Documentation drift after a fix is itself a §11.4 violation. (d) **No-bluff certification per cycle.** Before tagging: `meta_test_false_positive _proof.sh` returns all gates green AND every gate's paired mutation FAILs (no bluff gates); `docs/Issues.md` open-set is empty or every entry explicitly classified out-of-scope-for-this-tag with operator sign-off (no known issues hidden); full suite returns zero new FAILs on either device (no working feature regressed); every gate has a paired mutation; every test produces positive evidence; every assertion catches its own negation (no error-prone or bluff-proof leftover).

Non-compliance is a release blocker regardless of context.

§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim, 2026-05-07):

> "We MUST HAVE still analyzing of recorded materials and comprehensive validation and verification for issues we used to test! For example if there is audio at all or video, if so, is it good and proper or is it faulty? Does it have glitches, frame issues and other possible obstructions? IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence. A test that captures a 0-byte mp4 (Bug #24) and PASSes because "the recording file exists" is the exact PASS-bluff pattern §11.4 forbids. Content-quality analysis is what closes that gap.

Audio quality analysis — every audio test that PASSes MUST verify ALL of: (1) **Presence** — non-trivial RMS amplitude in captured WAV / `/proc/asound/.../pcm*p/sub0/hw_params`. (2) **Channel count** — `ffprobe -show_streams` matches the test's claim (2.0 / 5.1 / 7.1). (3) **Sample rate + bit depth** — match the codec / pipeline under test. (4) **Glitch census** — XRUN / FastMixer underrun-overrun-partial / AudioFlinger writeError counts above tolerance MUST classify explicitly (PASS within budget, WARN above, FAIL on hard limits per §11.4.1 SKIP-vs-FAIL decision tree). (5) **Coexistence-artifact census** — for tests that exercise WiFi/BT alongside audio: BT TX queue overflow, A2DP src underflow, coex notification storms, 2.4 GHz radio contention.

Video quality analysis — every video test that PASSes MUST verify ALL of: (1) **Presence** — captured screen recording has non-zero file size AND `ffprobe -count_frames` reports decoded-frame total > 0. 0-byte mp4 (Bug #24) is the canonical PASS-bluff and triggers §11.4.4 STOP. (2) **Routing target** — analyzer + action-timeline confirms video appeared on the *intended* display (primary vs secondary HDMI; Bug #13 pattern). (3) **Frame health** — drop count, frame-time variance (jitter), freeze detection (SSIM > 0.99 for ≥ 1 s), tearing. (4) **Obstruction census** — Tesseract OCR scan for hostile overlays (`Application not responding`, `Force close`, sign-in dialog, geo-restriction overlay, ad break, paywall, `App is not certified`). (5) **Resolution + codec** — captured frame dimensions match the test's claim; downgrade is a PASS-bluff.

Challenges (HelixQA) are bound equally — every Challenge that asserts PASS MUST run all five audio + five video layers. A Challenge that scores PASS without applicable analysis is the same class of defect as a unit test that does.

Tooling guarantee: audio = `tinycap` + `aplay --dump-hw-params` + `ffprobe` + `/proc/asound` parsers (`lib/audio_validation.sh` per §11.2.5). Video = `screenrecord` + `ffprobe -count_frames` + `recording-analyzer` + Tesseract OCR (`scripts/dual_display_record.sh`

- `cmd/recording-analyzer/` per §11.4.2.A and §11.4.2.C). Tests dispatched against video evidence MUST honor §11.4.4 test-interrupt-on-discovery when the analyzer reports empty input — do not silently absorb that as a generic PASS-bluff banner.

Non-compliance is a release blocker regardless of context.

Lava Sixth Law inheritance (consumer-side anchor, 2026-04-29)

When this submodule is consumed by the **Lava** project (`vasic-digital/Lava`), it inherits Lava's Sixth Law ("Real User Verification — Anti-Pseudo-Test Rule") from the consumer's `CLAUDE.md`. Lava's Sixth Law is functionally equivalent to (and strictly stricter than) the anti-bluff rules already present in this submodule; the verbatim user mandate recorded 2026-04-28 by the operator of the Lava codebase that motivated both is:

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

The 2026-04-29 lessons-learned addenda recorded in Lava's `CLAUDE.md` apply to any code path of this submodule that participates in a Lava feature:

- **6.A — Real-binary contract tests.** Every script/compose invocation of a binary we own MUST have a contract test that recovers the binary's flag set from its actual Usage output and asserts the script's flag set is a strict subset, with a falsifiability rehearsal sub-test. Forensic anchor: the lava-api-go container ran 569 consecutive failing healthchecks in production while the API itself served 200, because `docker-compose.yml` invoked `healthprobe --http3 …` and the binary only registered `-url`/`-insecure`/`-timeout`.

- **6.B — Container "Up" is not application-healthy.** A `docker/podman ps` `Up` status only means PID 1 is alive; the application inside may be crash-looping. Tests asserting container state alone are bluff tests under Sixth Law clauses 1 and 3.
- **6.C — Mirror-state mismatch checks before tagging.** "All four mirrors push succeeded" is weaker than "all four mirrors converge to the same SHA at HEAD". `scripts/tag.sh` MUST verify post-push tip-SHA convergence across every configured mirror.

Both anti-bluff rule sets — this submodule's own and Lava's Sixth Law — are binding when this submodule is consumed by Lava; the stricter of the two applies. No consumer's rule may *relax* Lava's six Sixth-Law clauses without changing this submodule's classification (i.e. demoting it from Lava-compatible).

Lava Seventh Law inheritance (Anti-Bluff Enforcement, 2026-04-30)

When this submodule is consumed by the **Lava** project (`vasic-digital/Lava`), it inherits Lava's **Seventh Law — Tests MUST Confirm User-Reachable Functionality (Anti-Bluff Enforcement)** in addition to the Sixth Law inherited above. The Seventh Law was added to Lava's `CLAUDE.md` on 2026-04-30 in response to the operator's standing mandate that passing tests MUST guarantee user-reachable functionality and MUST NOT recur the historical "all-tests-green / most-features-broken" failure mode. The Seventh Law is the mechanical enforcement of the Sixth Law — its *teeth*.

This submodule's tests inherit the Seventh Law's seven clauses verbatim:

1. **Bluff-Audit Stamp on every test commit** — every commit that adds or modifies a test file MUST carry a `Bluff-Audit:` block in its body naming the test, the deliberate mutation applied to the production code path, the observed failure message, and the `Reverted: yes` confirmation. Pre-push hooks reject test commits that lack the stamp.
2. **Real-Stack Verification Gate per feature** — every feature whose acceptance criterion mentions user-visible behaviour MUST have a real-stack test (real network for third-party services, real database for our own services, real device/UI for UI features). Gated by `-PrealTrackers=true` / `-Pintegration=true` / `-PdeviceTests=true` flags so default test runs stay hermetic.
3. **Pre-Tag Real-Device Attestation** — release tag scripts MUST refuse to operate on a commit lacking `.lava-ci-evidence/<tag>/`

`real-device-attestation.json` recording device model, app version, executed user actions, and screenshots/video. There is no exception.
4. **Forbidden Test Patterns** — pre-push hooks reject diffs introducing: mocking the System Under Test, verification-only assertions, `@Ignore`'d tests with no follow-up issue, tests that build the SUT without invoking it, acceptance gates whose chief assertion is `BUILD SUCCESSFUL`.
5. **Recurring Bluff Hunt** — once per development phase, 5 random `*Test.kt` / `*_test.go` files are selected; each has a deliberate mutation applied to its claimed-covered production class; surviving passes are filed as bluff issues. Output recorded under `.lava-ci-evidence/bluff-hunt/<date>.json`.
6. **Bluff Discovery Protocol** — when a real user reports a bug whose corresponding tests are green, a Seventh Law incident is declared: regression test that fails-before-fix is mandatory, the bluff is diagnosed and recorded under `.lava-ci-evidence/sixth-law-incidents/<date>.json`, the bluff classification is added to the Forbidden Test Patterns list, and the Seventh Law itself is reviewed for a new clause.
7. **Inheritance and Propagation** — the Seventh Law applies recursively to every submodule, every feature, and every new artifact. Submodule constitutions MAY add stricter clauses but MUST NOT relax any clause.

The authoritative verbatim text lives in the parent Lava `CLAUDE.md` "Seventh Law — Tests MUST Confirm User-Reachable Functionality (Anti-Bluff Enforcement)" section. Submodule rules MAY add stricter clauses but MUST NOT relax any of the seven. Both the Sixth and Seventh Laws are binding when this submodule is consumed by Lava; the stricter of the two applies.

Clauses 6.I and 6.J (added 2026-05-04, inherited per 6.F)

- **Clause 6.I — Multi-Emulator Container Matrix as Real-Device Equivalent** — see root `/CLAUDE.md` §6.I. Real-device verification, where this submodule's work requires it (per 6.G clause 5 / Sixth Law clause 5 / Seventh Law clause 3), is satisfied ONLY by the project's container-bound multi-emulator matrix: minimum API 28, 30, 34, latest stable; minimum phone + tablet + (where applicable) TV; per-AVD attestation rows in `.lava-ci-evidence/<tag>/real-device-verification.md`; cold-boot for the gate; falsifiability rehearsal per Android-version-class. A single passing emulator (or device) is NOT the gate — the matrix is.
- **Clause 6.J — Anti-Bluff Functional Reality Mandate** — see root `/CLAUDE.md` §6.J. Every test, every Challenge Test, and every CI gate touched by this submodule MUST do exactly one job: confirm the feature it claims to cover actually works for an

end user, end-to-end, on the gating matrix. CI green is necessary, never sufficient. Adding a test the author cannot execute against the gating matrix is itself a bluff. Tests must guarantee the product works — anything else is theatre.

Clauses 6.K and 6.L (added 2026-05-04, inherited per 6.F)

- **Clause 6.K — Builds-Inside-Containers Mandate** — see root `/CLAUDE.md` §6.K. Every release-artifact build MUST run inside the project's container-bound build path (anchored on `vasic-digital/Containers`'s build orchestration: `cmd/distributed-build` + `pkg/distribution` + `pkg/runtime`), not on the developer's bare host. Local incremental dev builds on the host are permitted for iteration; the gate, the release-artifact build, and the build whose output goes through the emulator matrix (clause 6.I) MUST go through Containers. The accompanying 6.K-debt entry tracks the package additions (`pkg/emulator/`, `pkg/vm/`) that are owed.
- **Clause 6.L — Anti-Bluff Functional Reality Mandate (Operator's Standing Order)** — see root `/CLAUDE.md` §6.L. Every test, every Challenge Test, every CI gate has exactly one job: confirm the feature works for a real user end-to-end on the gating matrix. CI green is necessary, never sufficient. Tests must guarantee the product works — anything else is theatre. The operator has invoked this mandate TWENTY-THREE TIMES across two working days; the repetition itself is the forensic record. The 10th invocation (2026-05-05, immediately after Phase 7 readiness was reported, when the operator commissioned the full rebuild-and-test-everything cycle for tag Lava-Android-1.2.3): "Rebuild Go API and client app(s), put new builds into releases dir (with properly updated version codes) and execute all existing tests and Challenges!". If you find yourself rationalizing a "small exception" — STOP. There are no small exceptions. The Internet Archive stuck-on-loading bug, the broken post-login navigation, the credential leak in C2, the bluffed C1-C8 — these are what "small exceptions" produce.

Clause 6.M (added 2026-05-04 evening, inherited per 6.F)

- **Clause 6.M — Host-Stability Forensic Discipline** — see root `/CLAUDE.md` §6.M. Every perceived-instability event during a session that touches this submodule MUST be classified into Class I (verifiable host event), Class II (resource pressure), or Class III (operator-perceived without forensic evidence) AND audited via the 7-step forensic protocol (uptime+who, journalctl logind events, kernel critical events, free -h, df -h, forbidden-command grep across tracked files, container state inventory). Findings recorded under `.lava-ci-evidence/sixth-law-incidents/<date>-<slug>.json`. **Container-runtime safety analysis (recorded once in root §6.M, referenced forever):** rootless Podman has NO host-level power-management privileges; rootful Docker is not installed on the operator's primary host. Container operations cannot cause Class I host events on the audited host configuration. A perceived-instability event without an audit record is itself a Seventh Law violation under clause 6.J ("tests must guarantee the product works" — applied recursively to incident response).

Clause 6.N (added 2026-05-05, inherited per 6.F)

- **Clause 6.N — Bluff-Hunt Cadence Tightening + Production Code Coverage** — see root `/CLAUDE.md` §6.N. Beyond the Seventh Law clause 5 baseline (5 random `*Test.kt` files every 2-4 weeks), bluff hunts now fire IN-cycle on three triggers: (1) per operator anti-bluff-mandate invocation — first/day full 5+2, subsequent same-day lighter 1-2 file incident-response; (2) per matrix-runner/gate change (pre-push enforced via §6.N-debt — owed); (3) per phase-gating attestation file added (pre-push enforced via §6.N-debt — owed). Bluff hunts MUST also sample production code: 2 files per phase from gate-shaping code (canonical list in root §6.N.2: `scripts/tag.sh` helpers, `scripts/check-constitution.sh`, `Submodules/Containers/pkg/emulator/`, `Submodules/Containers/cmd/emulator-matrix/`, the matrix runner's `writeAttestation` function) plus 0-2 from broader CI-touched code. Conceptual filter: "would a bug here be invisible to existing tests?". Forensic anchor: 2026-05-05 ultrathink-driven discovery of the 7-day-old `pkg/emulator/Boot()` port-collision bluff that was invisible to all existing test-only bluff hunts. §6.N-debt tracks the pre-push hook implementation owed via the Group A-prime spec (next brainstorming target).

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Forensic anchor — direct user mandate (verbatim):

> "We had to restart this session 3rd time in a row! The system of the host stays with no RAM memory for some reason! First make sure that whatever we do through our proced-

> ures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

The mandate. Project procedures MUST NOT use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

Three consecutive session-loss SIGKILLs on 2026-04-30 during 1.1.5-dev — every one happened while `scripts/build.sh` was running `m -j5` AOSP. Each Soong/Ninja job peaks at ~5–8 GiB RSS; collective RSS overran the 60% envelope and the kernel OOM-killer escalated, taking down `user@1000.service`. **§12.1's pre-flight check (refusing to start if host already distressed) was not enough** — the missing piece was an active CONSTRAINT on heavy work itself.

Mandatory protections (rock-solid):

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60 in `scripts/lib/host_session_safety.sh`.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100`.
3. `bounded_run` clamps `MemoryMax` down to the budget if the caller asks for more (cgroup-level enforcement via `systemd-run --user --scope -p MemoryMax=…`).
4. `host_safe_parallel_jobs` and `host_safe_build_jobs` return the safe `-j` count given an estimated per-job RSS, capped at `nproc`.
5. `scripts/build.sh` wraps `m -j` in `bounded_run`. If the build's collective RSS exceeds the budget, only the scope is OOM-killed; `user@<uid>.service` stays alive.

Captured-evidence enforcement. Pre-build gate `CM-MEMBUDGET-METATEST` locks all 7 invariants and fires every pre-build run.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits. Operators who need more headroom should reduce parallelism, close other workloads, or add RAM — NOT raise the percentage.

Canonical authority: parent [`docs/guides/ATMOSPHERE_CONSTITUTION.md`]() §12.6.

Non-compliance is a release blocker regardless of context.

§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)

Forensic anchor — direct user mandate (verbatim, 2026-05-08T18:30 MSK):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Tests, gates, status reports, closure narratives, commit messages, and operator-facing text MUST NOT use `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, or `appears to` when describing causes of failures, behaviour, or fix effectiveness. Either prove the cause with captured forensic evidence (logcat, dmesg, /sys readings, getprop, kernel ramoops, dropbox, strace, etc.) and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

Pre-build gate `CM-NO-GUESSING-MANDATE` greps recently-modified docs

- test scripts for the forbidden vocabulary outside explicit `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` blocks. Paired mutation introduces a `likely` token into a fresh status block → gate FAILs. Propagation gate `CM-COVENANT-114-6-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [`docs/guides/ATMOSPHERE_CONSTITUTION.md`] §11.4.6.

Non-compliance is a release blocker regardless of context.

§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)

A demotion from any FAIL classification (`OPEN`, `POSSIBLE PRODUCT DEFECT`, `FAIL`) to a lower-severity classification (`INVESTIGATED`, `MITIGATED`, `RESOLVED`, `WORKING-AS-INTENDED`) requires positive evidence captured under the **same conditions** that originally exposed the defect — same device, same firmware, same cycle position, same load profile.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged `UNCONFIRMED:` until same-conditions retest produces positive evidence. The expanded forbidden-vocabulary list:

Forbidden phrase	Why it bluffs
"isolated re-run PASSes therefore X was a flake"	Strips the very environment that exposed the defect.
"runtime drift"	Label for "we don't know what changed".
"intermittent" / "transient"	Label for "we don't know how to reproduce".

Forbidden phrase	Why it bluffs
"pending stress retest"	Defers the actual investigation indefinitely.
"correlates with X"	Hypothesis presented as causation.

Pre-build gate `CM-DEMOTION-EVIDENCE-RULE` scans Issues.md / Fixed.md / CONTINUATION.md for these phrases outside explicit `UNCONFIRMED:` / `UNATTRIBUTED:` / `PENDING_CYCLE_RETEST:` blocks. Propagation gate `CM-COVENANT-114-7-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.7.

Non-compliance is a release blocker regardless of context.

§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)

Before designing a non-trivial fix, implementing a new feature, or declaring an architectural choice, perform deep web research to verify the chosen approach is informed by current state-of-the-art. Research surface: official documentation (Android/AOSP/Khronos/CEA-861/AES/IEEE/IETF/ITU), vendor technical guides (Rockchip, Sipeed, Audinate Dante, Synaptics, Realtek, Bluetooth SIG), open-source codebases (Linux kernel, ALSA, Bluez, ExoPlayer, libVLC, MPV, FFmpeg, AOSP forks), coding tutorials + technical articles (Stack Overflow, AOSP Code Lab, AES papers), issue trackers (Android bug tracker, AOSP gerrit, GitHub issues).

A fix that re-invents a wheel — or reproduces a known-broken pattern — when the open-source community has already solved the problem is a §11.4 violation by omission. Every non-trivial fix's commit / Issues.md / Fixed.md entry MUST cite at least one external source URL OR the literal "NO external solution found — original work".

Pre-build gate `CM-RESEARCH-CITATION-PRESENT` scans new fix-direction blocks for the pattern. Propagation gate `CM-COVENANT-114-8-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Documentation continuity requirement: every fix landed under §11.4.8 also adds to `docs/guides/` a user-facing or developer-facing guide section where appropriate.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.8.

Non-compliance is a release blocker regardless of context.

§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)

When closing a multi-defect batch, all source-side fixes that DO NOT require runtime on-device validation to design MUST be landed BEFORE the next firmware rebuild. Anti-pattern eliminated: `Fix A → rebuild → flash → cycle → fix B → rebuild → ...` serializes 7-8 hours per fix instead of batching all into ONE build cycle. Operator time is the scarce resource.

Exceptions documented in commit message as `REQUIRES_REBUILD: <reason>`: kernel-5.10/ changes, atmosphere-*.sh boot-script side-effects, hardware/rockchip/ HAL behavior — each gates downstream state and requires firmware to validate.

Before declaring a batch "ready for rebuild": pre-build GREEN + meta-test GREEN + existing-device validations performed where possible + Issues.md/Fixed.md/CONTINUATION.md in sync (+ HTML/PDF exported) + §11.4.8 research citations all logged.

Propagation gate `CM-COVENANT-114-9-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.9.

Non-compliance is a release blocker regardless of context.

§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)

All credentials, secrets, API tokens, passwords, phone numbers, OAuth tokens, signing keys MUST NEVER live in tracked files. Templates with placeholder values are allowed (`.example` suffix). Tests load credentials at runtime from `scripts/testing/secrets/` (or per-submodule equivalent); operator-populated files are `chmod 600`, directory is `chmod 700`. `.env`, `.env.*`, `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + `README.md` exception) git-ignored project-wide.

Test scripts MUST NEVER echo credentials to stdout/stderr/logcat. Screen- recording of sign-in flows MUST redact credential-bearing frames. Per-service file separation (`.netflix.env`, `.disney.env`, etc.) limits blast radius.

Forensic-rotation policy: suspected leak → rotate at provider, update local `.env`, audit captured artifacts. Pre-build gate `CM-CREDENTIAL-LEAK-SCAN` greps tracked files for entropy-suspicious password strings + known API-token formats. Propagation gate

CM-COVENANT-114-10-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.10.

Non-compliance is a release blocker regardless of context.

§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)

Every test that issues `am start` / `cmd media_session play` / `MediaController.play` MUST issue matching `am force-stop` / `input keyevent KEYCODE_MEDIA_STOP` + register cleanup in `EXIT` trap. Verified via positive evidence (Arvus codec-state → `N.E.`, `dumpsys media_session` shows no PLAYING for test app). `test_all_fixes.sh` post-test sanity check FAILs the just-completed test if it left orphan playback. HelixQA Challenges bound equally. No grace period — "next test will clean it up" is §11.4 PASS-bluff.

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.14. Pre-build gates `CM-TEST-PLAYBACK-CLEANUP` + `CM-COVENANT-114-14-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)

Every active item in `docs/Issues.md` carries a `**Status:**` line with one of six values: `Queued`, `In progress`, `Ready for testing`, `In testing`, `Reopened`, `Fixed (→ Fixed.md)`. Status MUST be updated as the item progresses through its lifecycle. `Fixed` requires captured-evidence per §11.4.5 + migration to Fixed.md.

The auto-generated `docs/Issues_Summary.md` includes the Status column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by `CM-DOCS-EXPORT-SYNC` (§11.4.12 + §11.4.15 amendment).

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.15. Pre-build gates `CM-ITEM-STATUS-TRACKING` + `CM-COVENANT-114-15-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)

Every active item in `docs/Issues.md` carries a `**Type:**` line with one of three values: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The vocabulary is CLOSED — no other value is permitted.

The auto-generated `docs/Issues_Summary.md` includes the Type column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by `CM-DOCS-EXPORT-SYNC` (§11.4.12 + §11.4.15 + §11.4.16 amendment).

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md]() §11.4.16. Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION`.

Non-compliance is a release blocker regardless of context.

§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)

Whenever an HDMI sink with a network-accessible introspection API is present (current example: Arvus H2-4D-273 at `http://192.168.4.185/`), the test suite MUST consume the sink's report as captured-evidence for every audio test asserting a codec / channel-count / passthrough mode. On-SoC HAL telemetry ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Reference: `scripts/testing/lib/arvus_probe.sh`, `scripts/testing/arvus_probe.sh`, `docs/guides/ARVUS_HDMI_INTEGRATION.md`. Pre-build gate `CM-ARVUS-EVIDENCE-INTEGRATED` (7 invariants) + paired mutation. No hardcoding (env: `ARVUS_HOST` etc.). Topology dispatch per §11.4.3 — sink unreachable → SKIP, never FAIL. Identity verification (MAC match) before consuming codec-state. Anti-stickiness post-stop. HelixQA Challenges bound equally.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md]() §11.4.13. Integration reference: `docs/guides/ARVUS_HDMI_INTEGRATION.md`.

Non-compliance is a release blocker regardless of context.

§11.4.11 — File-layout discipline (User mandate, 2026-05-12)

Files live in canonical directories per type:

- Shell scripts → `scripts/` (legacy: `scripts/legacy/`)
- Log files → `logs/` (legacy: `logs/legacy/`)
- Release artifacts → `releases/<app>/<version>/`
- Operator credentials → `scripts/testing/secrets/` (per §11.4.10, git-ignored)
- Markdown docs → `docs/` + `docs/guides/` + `docs/research/` + `docs/superpowers/plans/`
- Per-version changelogs → `docs/changelogs/`

- Hardware ID photos → `docs/hardware/<device-slug>/`

Repo root contains ONLY: AOSP-mandated top-level files (Android.bp, Makefile, bootstrap.bash, BUILD, kokoro, lk_inc.mk, OWNERS, version_defaults.mk), project metadata (README/CLAUDE/AGENTS/CONTRIBUTING/LICENSE/NOTICE/VERSION), dot-files (.gitignore/.gitmodules), and standard top-level dirs (build/, device/, external/, frameworks/, hardware/, kernel-5.10/, packages/, prebuilts/, scripts/, system/, tools/, vendor/, docs/, releases/, logs/).

NO bash scripts in repo root except AOSP-mandated `bootstrap.bash`. NO log files in repo root. NO duplicate filenames between root and `scripts/`. NO release artifacts in root. Moves require triple-verification (audit all references + distinguish absolute vs subdir-local + confirm no AOSP build- system requirement). Pre-build gate `CM-FILE-LAYOUT-DISCIPLINE` enforces. Propagation gate `CM-COVENANT-114-11-PROPAGATION` enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.11.

Non-compliance is a release blocker regardless of context.

§11.4.12 — Issues_Summary.md sync mandate (User mandate, 2026-05-12)

docs/Issues_Summary.md is the canonical short-form summary of all open items. MUST be regenerated + re-exported (HTML + PDF) whenever Issues.md changes. Generator: scripts/testing/generate_issues_summary.sh. Pre-build gates `CM-ISSUES-SUMMARY-SYNC` + `CM-COVENANT-114-12-PROPAGATION` enforce mechanically.

Sort order (User mandate refinement 2026-05-12): severity DESC (C → M → L), then intra-group criticality DESC inside each group. Most critical row = #1, least critical = #N. Documented at the top of the generated file.

Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` — runs generator + `export_progress_docs.sh` in one shot. MUST be invoked after any edit to Issues.md or Issues_Summary.md. HTML+PDF exports are NEVER manually invoked; they ALWAYS travel with the markdown.

Canonical authority: parent `docs/guides/ATMOSPHERE_CONSTITUTION.md` §11.4.12.

Non-compliance is a release blocker regardless of context.

Clause 6.O (added 2026-05-05, inherited per 6.F)

- **Clause 6.O — Crashlytics-Resolved Issue Coverage Mandate** — see root `/CLAUDE.md` §6.O. Every Crashlytics-recorded issue (fatal OR non-fatal) closed/resolved by any commit MUST gain (a) a validation test in the language of the crashing surface that reproduces the conditions, (b) a Challenge Test under `app/src/androidTest/kotlin/lava/app/challenges/` (client) or `tests/e2e/` (server) that drives the same user-facing path, and (c) a closure log at `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` recording the issue ID, root-cause analysis, fix commit SHA, and links to the tests. `scripts/tag.sh` MUST refuse release tags whose CHANGELOG mentions Crashlytics fixes without matching closure logs. Marking a Crashlytics issue "closed" in the Console requires the test coverage to land first — never close-mark before the regression-immunity tests exist. Forensic anchor: 2026-05-05, 2 Crashlytics-recorded crashes within minutes of the first Firebase-instrumented APK distribution (Lava-Android-1.2.3-1023, commit `e9de508`); post-mortem at `.lava-ci-evidence/crashlytics-resolved/2026-05-05-firebase-init-hardening.md`. The operator's ELEVENTH §6.L invocation made this clause load-bearing.

Clause 6.P (added 2026-05-05, inherited per 6.F)

- **Clause 6.P — Distribution Versioning + Changelog Mandate** — see root `/CLAUDE.md` §6.P. Every distribute action (Firebase App Distribution, container registry pushes, releases/snapshots, scripts/tag.sh) MUST: (1) carry a strictly increasing versionCode (no re-distribution of already-published codes); (2) include a CHANGELOG entry — canonical file `CHANGELOG.md` at repo root + per-version snapshot at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md`; (3) inject the changelog into the App Distribution release-notes via `--release-notes`. `scripts/firebase-distribute.sh` REFUSES to operate when current versionCode ≤ last-distributed versionCode for the channel, OR when CHANGELOG.md lacks an entry for the current version, OR when the per-version snapshot file is missing. `scripts/tag.sh` enforces the same gates pre-tag. Re-distributing the same versionCode is forbidden across distribute sessions; idempotent retry within a single session is permitted. Forensic anchor: 2026-05-05 23:11 operator's TWELFTH §6.L invocation: "when distributing new build it must have version code bigger by at least one then the last version code available for download (already distribited). Every distributed build MUST CONTAIN changelog with the details what it includes compared to previous one we have published!"

Clause 6.Q (added 2026-05-05, inherited per 6.F)

- **Clause 6.Q — Compose Layout Antipattern Guard** — see root `/CLAUDE.md` §6.Q. Forbids nesting vertically-scrolling lazy layouts (LazyColumn, LazyVerticalGrid, LazyVerticalStaggeredGrid) inside parents giving unbounded vertical space (verticalScroll, unbounded wrapContentHeight, LinearLayout-with-weight wrapper). Equivalent rule horizontally for LazyRow / LazyHorizontalGrid / LazyHorizontalStaggeredGrid. Per-feature structural tests + Compose UI Challenge Tests on the §6.I matrix are the load-bearing acceptance gates. Forensic anchor: 2026-05-05 23:51 operator-reported "Opening Trackers from Settings crashes the app" — TrackerSelectorList used LazyColumn nested in TrackerSettingsScreen's Column(verticalScroll). Closure log: `.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md`. Pattern guard: `feature/tracker_settings/src/test/.../TrackerSelectorListLazyColumnRegressionTest.kt`. The operator THIRTEENTH §6.L invocation triggered this clause.

§6.R — No-Hardcoding Mandate (inherited 2026-05-06, per §6.F)

See root `/CLAUDE.md` §6.R. No connection address, port, header field name, credential, key, salt, secret, schedule, algorithm parameter, or domain literal in tracked source code. Every such value MUST come from `.env` (gitignored), generated config, runtime env var, or mounted file. Submodule MAY add stricter rules but MUST NOT relax.

§6.S — Continuation Document Maintenance Mandate (inherited 2026-05-06, per §6.F)

See root `/CLAUDE.md` §6.S. The file `docs/CONTINUATION.md` (in the parent Lava repo) is the single-file source-of-truth handoff document for resuming work across any CLI session. Every commit that changes phase status, lands a new spec/plan, bumps a submodule pin, ships a release artifact, discovers/resolves a known issue, or implements an operator scope directive MUST update `docs/`

`CONTINUATION.md` in the SAME COMMIT. The §0 "Last updated" line MUST track HEAD. Submodule MAY add stricter rules (e.g., maintain its own CONTINUATION) but MUST NOT relax this clause.

Anti-Bluff and Quality Mandate

Article XI §11.9 — Anti-Bluff Forensic Anchor

> Verbatim user mandate: "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Operative rule: Every PASS MUST carry positive runtime evidence. No false-success results are tolerable.

Bluff Taxonomy: wrapper, contract, structural, comment, skip.

§6.T — Universal Quality Constraints (inherited 2026-05-06, per §6.F)

See root `/CLAUDE.md` §6.T. All four sub-points (Reproduction-Before-Fix, Resource Limits for Tests & Challenges, No-Force-Push, Bugfix Documentation) apply verbatim. This submodule MAY add stricter rules but MUST NOT relax any of §6.T.1–§6.T.4.

§6.U — No sudo/su Mandate (inherited 2026-05-08, per §6.F)

See root `/CLAUDE.md` §6.U. Every use of `sudo` or `su` is strictly forbidden. Operations requiring elevated privileges MUST use container-based solutions from the `vasic-digital/Containers` submodule or be provided by local project/Submodule dependencies that build automatically. The pre-push hook rejects files containing `sudo ` or `su ` patterns. This submodule MAY add stricter rules but MUST NOT relax.

§6.V — Container Emulators Mandate (inherited 2026-05-08, per §6.F)

See root `/CLAUDE.md` §6.V. Every Android emulator instance for Challenge Tests / UI verification MUST run inside a container managed by the `vasic-digital/Containers` submodule. Rootless Podman/Docker only. All tests execute inside containers. The §6.I matrix (API 28/30/34/latest, phone/tablet/TV) runs inside container-bound emulators. This submodule MAY add stricter rules but MUST NOT relax.

§6.W — GitHub + GitLab Only Remotes (inherited 2026-05-08, per §6.F)

See root `/CLAUDE.md` §6.W. Only GitHub (`vasic-digital/*`, `HelixDevelopment/*`) and GitLab (`vasic-digital/*`, `HelixDevelopment/*`) are permitted as Git remotes. GitFlic, GitVerse, and all other providers are forbidden. The 4-mirror model is replaced by 2-mirror (GitHub + GitLab). This submodule MAY add stricter rules but MUST NOT relax.

§6.X — Container-Submodule Emulator Wiring Mandate (inherited 2026-05-13, per §6.F)

See root `/CLAUDE.md` §6.X. Every Android emulator instance the project depends on for testing MUST execute its emulator process INSIDE a podman/docker container managed by `Submodules/Containers/`, NOT be host-direct-launched by Containers-submodule code that runs on the host. The Containers submodule's `pkg/runtime/` (rootless podman/docker auto-detection) brings the container up; `pkg/emulator/` orchestrates the AVD lifecycle inside it. Lava-side `scripts/run-emulator-tests.sh` is thin glue forwarding to the Containers CLI. The container-bound path is the gate — host-direct emulators are permitted for workstation iteration only. §6.X-debt tracks the wiring implementation owed to `Submodules/Containers/`. This submodule MAY add stricter rules but MUST NOT relax.

Submodule Decoupling & Reusability — MANDATORY

This repository is **shared infrastructure** consumed by multiple independent consumer projects. Its specialized responsibility makes it reusable — and that reusability is destroyed the moment any consumer's specifics leak in.

Hard rules when editing anything in this repository:

- DO NOT hardcode any specific consumer project's name, platform list, paths, version strings, or release-naming conventions.
- DO NOT import / reference any consumer-project namespace.
- DO NOT embed consumer-project-specific governance, branding, or rule numbering in `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md`.
- DO assume N ≥ 2 unrelated consumer projects exist, even if you only know of one today.

Cross-project rules MUST be phrased generically ("every consuming project's full platform matrix"), never with a specific consumer's matrix hardcoded.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

> Verbatim user mandate (2026-05-14): "*Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!*"

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

§6.Z — Anti-Bluff Distribute Guard (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.Z. No artifact may be distributed (Firebase App Distribution, Google Play Store release, container image push, this submodule's binary release, any future channel) UNLESS the corresponding end-to-end tests have been **EXECUTED — not source-compiled, EXECUTED** — against the EXACT artifact about to be distributed, AND have **passed**. Pre-distribute test-evidence file required at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>-test-evidence.{md,json}` with matching commit SHA, timestamp within 24h, `BUILD SUCCESSFUL` (or per-language pass marker) verbatim in captured output. Cold-start verification is the load-bearing canary. Distributing a faulty version is a constitutional violation by construction. §6.Z-debt is open: mechanical enforcement via `scripts/firebase-distribute.sh` Phase 1 Gate 6 + pre-push hook check is documented but not yet enforced. Forensic anchor: 2026-05-14 Galaxy S23 Ultra cold-launch crash on Lava-Android-1.2.19-1039 (Crashlytics `40a62f97a5c65abb56142b4ca2c37eeb` — `painterResource()` rejection of `<layer-list>` drawable); agent had skipped Compose UI test execution citing the wrong §6.X caveat. Operator's 26th §6.L invocation: "Anti-bluff policy MUST BE ENFORCED ALWAYS!!!" This submodule MAY add stricter rules but MUST NOT relax this clause.

§6.AA — Two-Stage Distribute Mandate (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.AA. When an artifact has both a debug and a release variant (or analogous dev-vs-prod build types — including this submodule's binary release if it ships separate dev / prod variants), distribute MUST happen in TWO STAGES with operator-confirmed verification between them. Stage 1 distributes the debug / dev variant only; the operator verifies the **distributed** debug variant on the failure-surface device class. Stage 2 distributes the release / prod variant only ONLY AFTER written stage-1 verification, with the §6.Z test-evidence file appended with a `release-stage` section. No combined distribute permitted by default; the combined path requires explicit per-cycle operator authorization recorded in the evidence file. The R8 / minification surprise class on Android (or analogous stripping / production-only optimization classes on other artifacts) is the load-bearing reason. §6.AA-debt is open: mechanical enforcement via `scripts/firebase-

`distribute.sh` default flip + refusal of out-of-order `--release-only` + paired `last-version-{debug,release}` per-channel pre-push check is documented but not yet enforced. Forensic anchor: 2026-05-14 operator directive immediately after the §6.Z forensic-anchor crash on Lava-Android-1.2.19-1039: "for purposes like this one we shall distribute via Firebase DEV / DEBUG version only. Once we try it, you continue and once all verified you distribute RELEASE too!" This submodule MAY add stricter rules but MUST NOT relax this clause.

§6.AB — Anti-Bluff Test-Suite Reinforcement (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.AB. Every existing test + Challenge in this submodule MUST be auditable for the anti-bluff property "would this test fail if the user-visible behavior broke in a way a real user would notice?" Per-feature completeness checklist: rendering correctness (assert dominant color matches expected hue, not just RGB-variance), state-machine completeness (negative tests for forbidden transitions), gating logic (gate fires only on actual completion criterion). Bluff-hunt cadence escalation: every defect not caught by an existing test triggers a 5-file defect-driven hunt of adjacent tests, recorded under `.lava-ci-evidence/bluff-hunt/<date>-defect-driven-<slug>.json`. Discrimination test mandatory per Challenge Test: deliberately-broken-but-non-crashing production code MUST cause the Challenge Test to fail. Forensic anchor: 2026-05-14 Lava-Android-1.2.20-1040 white-icon + onboarding-gate-bypass — both passed all existing tests but failed for the user. Operator's 27th §6.L invocation: "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!" This submodule MAY add stricter rules but MUST NOT relax this clause.

§6.AC — Comprehensive Non-Fatal Telemetry Mandate (inherited 2026-05-14, per §6.F)

See root `/CLAUDE.md` §6.AC. Every catch / error / fallback / unexpected-state path on every distributable artifact in this submodule MUST record a non-fatal telemetry event with sufficient context to triage the failure remotely. The Android-side canonical entry is `analytics.recordNonFatal(throwable, ctx)` / `analytics.recordWarning(message, ctx)` (lava.common.analytics.AnalyticsTracker); the Go-side equivalent is `observability.RecordNonFatal(ctx, err, attrs)`. Cancellation throwables are filtered automatically. Mandatory context: feature/module + operation + error_class + error_message (truncated 1024, no credentials per §6.H) + per-platform extras. Forbidden: silent fallbacks without telemetry; credentials/tokens/cookies/PII unredacted in event attributes. §6.AC-debt is open: Detekt + Go-vet rules flagging `try/catch` blocks lacking the telemetry call, pre-push hook integration. Forensic anchor: 2026-05-14 operator: "Add comprehensive Crashlytics non-fatals recording all over the apps and API so we can track in the background all warnings, issues and unexpected situations!" This submodule MAY add stricter rules but MUST NOT relax this clause.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): "*Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!*"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-048 and constitution submodule `Constitution.md` §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): "*Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!*"

Before ANY modification to `constitution/{Constitution,CLAUDE,AGENTS}.md` in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never `git add -A`); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge `git submodule update --remote --init` + re-run cascade verifier (CONST-047); (7) bump consuming project's `.gitmodules` pointer to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-049 and constitution submodule `Constitution.md` §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): "*Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).*"

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources.

Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule's real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

Required dependency submodules (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): "*All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of*

> *the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!*"

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list` / `glab`):

(A) Equal-codebase. This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bugfix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

(B) Decoupling / reusability. This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

(C) Dependency-layout. Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org submodule chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

> Verbatim user mandate (2026-05-15): "*naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE re-*

> *named! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!*"

Every directory, submodule, and file in this submodule MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to every reference (configs, docs, source-code imports, governance files). Reference drift after rename = CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE language-roots; vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

`Upstreams/` → `upstreams/` transition: the constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH directory layouts; lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): regression test for reference resolution + full test-type matrix run + anti-bluff wire-evidence captured.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-052 and constitution submodule `Constitution.md` §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

> Verbatim user mandate (2026-05-15): "*every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!*"

Every project module, owned-by-us submodule, service, and application MUST ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.
2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder only — no real secrets even as examples), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.history/`, `.vscode/` (except shared settings).

Anti-bluff invariant: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by `CONST-053` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

> Verbatim user mandate (2026-05-15): "*We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone.*"

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.{recursive,conflict_resolution}`, `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by `CONST-054` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule `Constitution.md` §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

> Verbatim user mandate (2026-05-15): "*Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!*"

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: `Reopened`, Type: `Bug`); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by `CONST-055` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to

§11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

> Verbatim user mandate (2026-05-15): "*Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!*"

Every clone / add of a Git repository under HelixCode MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/`.zshrc`; implementation in the constitution submodule's `install_upstreams.sh` — already supports BOTH directory names since constitution commit `45d3678`) reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: the future `incorporate-submodule` per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

Cascade requirement: This anchor (verbatim or by `CONST-056` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal `**Status:**` value, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a CONST-057 violation. Gate `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by `CONST-057` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).

- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`. Gate `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by `CONST-058` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit `Lifted from <project> to constitution per §11.4.35` / `Demoted from constitution to <project> per §11.4.35` commit-message annotation.

Gate `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by `CONST-059` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

> Verbatim user mandate (2026-05-15): "*Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.*"

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false**

confidence — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by `CONST-060` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule `Constitution.md` §11.4.37 for the full mandate.

Anti-Bluff End-User Quality Guarantee (Escalated via HelixConstitution)

Canonical authority: `HelixConstitution/Constitution.md` §7.1 + §11.4.

Forensic anchor — verbatim operator mandate (2026-04-28):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

When writing a test in this submodule, ask: if every line of the unit under test were replaced with a trivial stub, would this test still pass? If yes, the test is bluff. Rewrite it to exercise the real behaviour.

Every PASS MUST carry positive runtime evidence. Consuming-project-specific evidence requirements are defined by each consuming project's Constitution.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between the batch's fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — complete retest is typically 12–48 hour elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composes with §11.4.4 (per-fix retest) — §11.4.40 is the additional final integrity check at RELEASE granularity. Composes with §11.4.7 — full-suite retest is the authoritative baseline for closures in the batch. No escape hatch — no `--skip-full-retest` or `--quick-release` flag exists.

Pre-build gate `CM-FULL-SUITE-RETEST-MANDATE` + paired mutation. Propagation gate `CM-COVENANT-114-40-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: constitution submodule `Constitution.md` §11.4.40.

Non-compliance is a release blocker regardless of context.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, or equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order): (1) `git fetch --all --prune --tags` against every configured remote — capture output. (2) Integrate every divergent commit locally via `git rebase` (local is strict superset), `git merge` (independent additions both deserve preservation), or operator-confirmed cherry-pick (remote subset already present locally). (3) Audit: no conflict markers (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty), no silent file drops (`git diff --stat HEAD@{1} HEAD`), every previously-passing test still passes per §11.4.4 / §11.4.40 baseline, every captured-evidence artifact still validates. (4) `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless §9.2 sub-clause 6 explicitly authorises it for a remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation.

Two-gate composition with CONST-043 — §11.4.41 does NOT relax CONST-043's operator-approval requirement. Gate A (CONST-043): operator types explicit per-operation force-push authorisation. Gate B (§11.4.41): agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push. Both gates required.

Verification artefact — every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing 7 elements: (i) `git fetch` output, (ii) per-remote `HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Composes with §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation), §11.4.32 (post-pull validation — audit step's mechanical companion), §11.4.37 (fetch-before-edit — step 1 enforces it for force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence requirement).

No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes. Pre-build gate `CM-COVENANT-114-41-PROPAGATION` enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs. Gate `CM-FORCE-PUSH-MERGE-FIRST` walks `docs/changelogs/<tag>.md` "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.41.

Non-compliance is a release blocker regardless of context.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18): "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: (a) programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + structured JSON result file); (b) headless intent dispatch + state poll (`am start --es` / `am broadcast` + `dumpsys` / `/proc/<pid>/maps` / `media.metrics` polling); (c) ADB-driven uiautomator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent); (d) network-side sink probe per §11.4.13; (e) HelixQA autonomous QA session per §11.4.27.

Coverage ledger (§11.4.25) classifies each feature as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` blocks release until migrated; cite tracked work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence + paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation coverage), §11.4.27 (no-fakes + 100% type coverage), §11.4.39 (per-feature on-device end-user validation), §11.4.43 (TDD RED-first), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach), §11.4.50 (deterministic consistency), §11.4.51 (live-ADB-first).

Pre-build gates: `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE`. Paired mutations. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule Constitution.md §11.4.52.

Non-compliance is a release blocker regardless of context.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z): "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, executable, emits markdown table with `Status` + `Type` columns per §11.4.19 column-alignment). Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH summaries in one shot, exports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary sibling — canonical pair), §11.4.19 (atomic Issues→Fixed migration trigger + column-alignment), §11.4.23 (colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` terminal values), §11.4.44 (revision header applies to `Fixed_Summary.md`), §12.10 (CONTINUATION.md resumption guarantee).

Pre-build gates: `CM-FIXED-SUMMARY-SYNC` (6 invariants — Fixed_Summary exists + HTML/PDF mtime ≥ md mtime + Fixed_Summary mtime ≥ Fixed mtime + generator + sync wrapper invokes generator) + `CM-COVENANT-114-53-PROPAGATION` (anchor literal across canonical files). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule Constitution.md §11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential Phase-chain. Each PWU has: ATM-NNN identifier (§11.4.54), Issues.md entry (§11.4.15+§11.4.16), file-scope manifest, §11.4.43 RED test, source patch, pre-build gate, post-flash test, paired §1.1 meta-test mutation, HelixQA Challenge bank entry, captured-evidence directory (§11.4.5+§11.4.52).

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in worktrees) → Stage 2 MERGE (serial conductor + §11.4.41 4-step merge-first) → Stage 3 REBUILD+FLASH (parallel where hardware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N.

Synchronization: 4-layer lock hierarchy (parent flock / per- submodule git / contention-path advisory locks for 10 forbidden cross-PWU paths / per-PWU worktree). Disjoint-scope PWUs fully parallel.

Anti-bluff merge-time enforcement (mandatory, all four): C1 §11.4.43 RED-test captured. C2 §1.1 paired meta-test mutation FAILs the gate. C3 §11.4.50 3-iter (or 10-iter) deterministic-consistency. C4 §11.4.5 captured-evidence per feature type. Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS REJECTED. HelixQA Challenge bank coverage MANDATORY for every user- visible PWU.

Phase 39.EX infrastructure gates (5 gates land the parallel infrastructure itself): `CM-PWU-PARALLEL-VALIDATION-ORCHESTRATOR`, `CM-PWU-HELIXQA-PER-DOMAIN-RUNNER`, `CM-PWU-WORKER-POOL-LOCKING`, `CM-PWU-FILE-SCOPE-PARTITION`, `CM-PWU-AUTO-MERGE-GATE-6CONDITIONS`. Each ships a paired meta-test mutation per §1.1.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE`

- `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION`. Paired mutations cover each gate. No escape hatch.

Canonical authority: constitution submodule `Constitution.md` §11.4.58. Project-specific implementation reference: `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their `docs/**/*.md`, `constitution/**/*.md`, owned HelixQA submodules' equivalents. Excludes: `external/**`, `prebuilts/**`, `packages/modules/**`, `kernel-5.10/**`, `out/**`, `build/**`, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be ≥ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION`. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): "any issue we spot must be fixed, bash scripts as well if they are broken!"

- "Make sure that this is mandatory rule!"

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh parses the whole script BEFORE executing, so the runtime `[ -n "${BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee …) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules). OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, <<<, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3) shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

CONST-062 / 065 / 066 / 067 / 075 / 076 / 077: Round-191 supplemental cascade — anchors §11.4.42, §11.4.45-47, §11.4.55-57

Anchors not covered by the Phase-39.EX cascade are added here for completeness per CONST-049 step 6.

- **CONST-062 / §11.4.42 — Iteration-discipline mandate.** Each fix cycle MUST run pre-build gate + post-build/post-flash test + paired §1.1 mutation + post-validation §11.4.40 retest. Truncating cycle to ship faster = violation. Subagents default to this path.
- **CONST-065 / §11.4.45 — Integration-Status-Doc Maintenance.** Every `Status.md` carries the §11.4.44 header and stays in sync with actual programme state at every commit advancing state. Out-of-sync = violation (CONST-044 severity).
- **CONST-066 / §11.4.46 — Validate-recent-work before post-flash sweep.** Each recent-work item since previous sweep MUST have its §11.4.43 RED test run against the live device and report GREEN before post-flash full-test sweeps. Skipping = violation.
- **CONST-067 / §11.4.47 — Firebase Data Review.** Every Firebase finding triaged per §11.4.47 severity table, deduped against Issues.md; new stacktrace gets §11.4.43 RED test before fix. Untriaged past SLA = violation.
- **CONST-075 / §11.4.55 — Reopens-history + per-item Reopens.md.** Every Reopened item gets `docs/reopens/ATM-NNN.md` tracking each cycle (date, source AI/User, reason from CONST-058 vocabulary, evidence path). `Reopens_Summary.md` regenerated on every reopen.
- **CONST-076 / §11.4.56 — Status_Summary parity + two-audience format.** `Status_Summary.md` (+ HTML/PDF exports) ships in operator-side + AI-side sections and stays in parity with underlying status docs at every commit advancing state.
- **CONST-077 / §11.4.57 — README.md doc-link section + revision metadata.** Every `README.md` carries (a) §11.4.44 revision header below H1, (b) Documentation link section listing canonical governance + status + plan docs.

For full mandate text + verbatim user quotes + gates, see constitution submodule `Constitution.md` §11.4.42, §11.4.45-47, §11.4.55-57.

Round-207 cascade — §11.4.59 + §11.4.60 (README always-sync + Documentation always-sync composite covenant)

> Verbatim user mandate (2026-05-19): "*fully review and update our main README document. ... Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML.*" AND (2026-05-19 ~09:00Z): "*Double check if all documents are properly tied with our root Constitution, CLAUDE.MD and AGENTS.MD so they are always up to date, always in sync and exported into PDF and HTML! ... Issues, Issues_Summary, Fixed, Fixed_Summary, Continuation, Status and Status_Summary for all contexts (areas) — THEY ALL MUST BE REGULARLY UPDATED, IN SYNC AND CONSISTENT without giving at any moment false picture about the state of the project or particular area(s) of it!*"

- **§11.4.59 — README always-sync mandate.** `README.md` at the project root is a §11.4.12-class always-sync document: kept current with every doc/integration/Status.md change, lockstep with `docs/CONTINUATION.md`, exported to `.html` + `.pdf` on every update via `scripts/testing/sync_readme_export.sh` (auto-invoked by `sync_issues_docs.sh`), carrying §11.4.44 revision header + Documentation Map section linking every Status / Status_Summary / spec / plan / guide / script-companion / changelog + the constitution submodule + per-audience navigation. Pre-build gate `CM-README-EXPORT-SYNC` enforces mtime parity (README.html + README.pdf ≥ README.md). Paired mutation backdates HTML+PDF → gate FAILs. No escape hatch — no `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag.
- **§11.4.60 — Documentation always-sync composite covenant.** Eight doc classes (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md, every Status_Summary.md) MUST be in sync at all times across `.md` + `.html` + `.pdf` artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; §11.4.60 binds them via single composite gate `CM-DOCS-COMPOSITE-SYNC` that FAILs the build if ANY instance's `.html` or `.pdf` mtime is older than `.md` mtime. Walks `docs/` recursively for Status fleet. Paired mutation backdates `docs/Issues.html` → gate FAILs. No escape hatch — no `--skip-composite-doc-sync`, `--allow-stale-html`, `--summary-not-applicable` flag exists.

Cascade requirement: These anchors (verbatim or by `§11.4.59` / `§11.4.60` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.59 + §11.4.60 for the full mandates.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio "82/84 PASS" + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.

- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §1.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post-grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

CONST-064: Mandatory Markdown metadata table + structured-doc ToC (cascaded from constitution submodule §11.4.61)

> Verbatim user mandate (2026-05-19): "*For every Markdown document which contains structured content (with headings / sections and sub-sections) make sure that every time we apply change to the structure, table of contents on the top of the document is created or updated! This is MANDATORY for every structured MARKDOWN document. Automatically its PDF and HTML versions MUST BE (re)generated! Introduce ... revision number, date and time of creation, date*

> *and time of last modification, other useful information we have in documents such as Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, Continuation and similar. ... make this mandatory for EVERY Markdown document from now on, update root constitution Submodule with these changes and commit and push it all to all upstreams.*"

> Verbatim 2026-05-19 operator mandate: "*all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!*"

Every tracked `*.md` in the §11.4.65 INCLUDED set MUST carry, immediately below the H1 title, a canonical Markdown metadata table with four MANDATORY rows (`Revision` — monotonic positive integer; `Created` — ISO 8601 date; `Last modified` — ISO 8601 date; `Status` — `active/draft/deprecated/superseded`) plus ENCOURAGED rows (`Status summary`, `Issues`, `Issues summary`, `Fixed`, `Fixed summary`, `Continuation`) rendered with `–`/`none` when N/A. Pre-existing §11.4.44 bold-line headers remain historically valid but MUST migrate to the table at the next substantive edit (30-day migration window from §11.4.61 landing). Any tracked `*.md` with **two or more H2 sections** MUST include a `## Table of contents` section immediately after the metadata table; the ToC MUST list every H2/H3 in document order with anchor links and MUST be regenerated on every structural change (heading added/removed/renamed/reordered). A stale ToC is a §11.4 PASS-bluff: operators reading the rendered Markdown see a navigation map that no longer matches the body. Narrow exemptions: `LICENSE`, `LICENSE.md`, `NOTICE`, `VERSION`, `OWNERS`, machine-generated `CHANGELOG.md`, plus the §11.4.65 EXCLUDED set. Anti-bluff gates `CM-MD-METADATA-PRESENT` (walks every tracked `*.md` minus exemptions; asserts H1 + 4 MANDATORY rows present within 25 lines below H1) + `CM-MD-TOC-PARITY` (for every `*.md` with ≥2 H2 sections asserts `## Table of contents` exists and its slugs match live H2/H3 set in order). Paired mutations strip metadata table → CM-MD-METADATA-PRESENT FAILs; rename one H2 without regenerating ToC → CM-MD-TOC-PARITY FAILs. The metadata or ToC change is a `.md` modification, which transitively triggers §11.4.65 regeneration of `.html`/`.pdf` siblings — the two clauses compose without duplicate enforcement.

Cascade requirement: This anchor (verbatim or by `CONST-064` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.61 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

> Verbatim user mandate (2026-05-20): "*We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!*"

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from `/proc/asound/.../status hw_ptr`; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence; a missing-but-required sink is `OPERATOR-BLOCKED` (release-blocker), never SKIP, never PASS. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists.

Cascade requirement: This anchor (verbatim or by `§11.4.68` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

> Verbatim user mandate (2026-05-20): "*Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!*"

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by `§11.4.70` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

> Verbatim user mandate (2026-05-20): "*before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!*"

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto-`--ours`/`--theirs`); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote` post-push. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by `§11.4.71` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

> Verbatim user mandate (2026-05-20): "*Make sure all fixes for audio are always top priority in main working stream!*"

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by `§11.4.72` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

> Verbatim user mandate (2026-05-20): "*Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!*"

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table `Revision` integer, bumped for every other change); (3) the metadata table MUST stay current (`Revision`, `Last modified`, `Status summary`, `Fixed`); (4) propagated copies of the rule MUST reference the active `specification.V<primary>.md`, not a stale archive; (5) on primary bump the old file moves to `<spec-dir>/archive/` with `Status: superseded`. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by `§11.4.73` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule `Constitution.md` §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

> Verbatim user mandate (2026-05-20): "*We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!*"

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — `vasic-digital` and `HelixDevelopment` on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a `Catalogue-Check:` field (`reuse <org/repo>@<sha>` / `extend <org/repo>@<sha>` / `no-match <date>`). Classification: universal.

Cascade requirement: This anchor (verbatim or by `§11.4.74` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule `Constitution.md` §11.4.74 for the full mandate.

§11.4.78 — CodeGraph code-intelligence mandate (cascaded from constitution submodule)

Inherited from `constitution/Constitution.md` §11.4.78. Every project worked on by AI coding agents — and every owned submodule when developed standalone — MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm package `@colbymchenry/codegraph`): a local SQLite semantic code-knowledge-graph exposed to AI agents over the Model Context Protocol (MCP), 100% local with no cloud or external API. Install globally via npm (no `sudo` — the npm prefix MUST be user-writable). Run `codegraph init` + `codegraph index`: `.codegraph/config.json` is tracked; `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `config.json` `exclude` list MUST exclude other-owned submodules and — non-negotiably — every §11.4.10 credential/secret path. Wire the `codegraph serve --mcp` MCP server into every CLI agent the developers use (Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`, Kimi CLI `~/.kimi/mcp.json`); every config references the bare `codegraph` command on `PATH`. Cover the integration with an anti-bluff verification suite whose per-agent end-to-end layer uses an unforgeable challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-runnable agents are documented SKIP gaps per §11.4.3, never faked PASSes. Document everything in `docs/CODEGRAPH.md`.

Cascade requirement: this anchor (verbatim or by §11.4.78 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md`, and `QWEN.md`. See the constitution submodule `Constitution.md` §11.4.78 for the full mandate. Non-compliance is a process violation.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Forensic anchor — direct user mandate (verbatim, 2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix or improvement landed in this project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load (N ≥ 100 iterations OR ≥ 30 s wall-clock; per-iteration latency p50/p95/p99 recorded) + concurrent contention (N ≥ 10 parallel invocations; no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input; every boundary produces a categorised result, never an uncaught exception).

Chaos (closed-set, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call; categorised recovery) + network-fault injection (drop/delay/reorder; `category=network|upstream` per §11.4.69) + input-corruption injection (corrupt .env / config / input file mid-test; detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion; refuse cleanly OR degrade gracefully — NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault; recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS cites a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration `latency.json`, `categorised_errors.txt`, `state_delta_snapshot.json`, `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`, each composing with `ab_pass_with_evidence` / `ab_skip_with_reason`. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in `trap '...' EXIT`; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (stress + chaos test files exist + executable + parseable under sh -n + bash -n per §11.4.67; helper library exists; the fix's pre-build gate cites the stress + chaos test file path) + paired meta-test mutation per §1.1 (stripping chaos-injection or per-iteration evidence capture → gate FAILs) + on-device test (if LIVE_ADB_TESTABLE per §11.4.51, dispatched against real device, evidence under `qa-results/<run-id>/stress_chaos/`) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden), §11.4.5 (captured-evidence quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag exists.